Description | Editorial | Solutions | Submissions

39. Combination Sum

Solved

Medium | Topics | Companies

Given an array of **distinct** integers `candidates` and a target integer `target`, return *a list of all **unique combinations** of* `candidates` *where the chosen numbers sum to* `target`. You may return the combinations in **any order**.

The **same** number may be chosen from `candidates` an **unlimited number of times**. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to `target` is less than `150` combinations for the given input.

Example 1:

```
Input: candidates = [2,3,6,7], target = 7
```

Code

Java | Auto

```java
class Solution {
    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        List<List<Integer>> result = new ArrayList<>();
        findCombinations(candidates, target, 0, new ArrayList<>(), result);
        return result;
    }

    private void findCombinations(int[] arr, int target, int index,
                                  List<Integer> list, List<List<Integer>> result) {

        if (target == 0) {
            result.add(new ArrayList<>(list));
            return;
        }

        for (int i = index; i < arr.length; i++) {
            if (arr[i] <= target) {
```

Saved

Ln 1, Col 1

Description | Accepted | Editorial | Solutions | Submissions

40. Combination Sum II

Solved

Medium | Topics | Companies

Given a collection of candidate numbers (`candidates`) and a target number (`target`), find all unique combinations in `candidates` where the candidate numbers sum to `target`.

Each number in `candidates` may only be used **once** in the combination.

Note: The solution set must not contain duplicate combinations.

Example 1:

```
Input: candidates = [10,1,2,7,6,1,5], target = 8
Output:
[
[1,1,6],
```

Code

Java | Auto

```java
class Solution {
    public List<List<Integer>> combinationSum2(int[] candidates, int target) {

        List<List<Integer>> result = new ArrayList<>();
        Arrays.sort(candidates);   // sort to handle duplicates

        backtrack(candidates, target, 0, new ArrayList<>(), result);
        return result;
    }

    private void backtrack(int[] arr, int target, int index,
                           List<Integer> list, List<List<Integer>> result) {

        if (target == 0) {
            result.add(new ArrayList<>(list));
            return;
        }
```

Saved | Ln 1, Col 1

Description | Accepted | Editorial | Solutions | Submissions

45. Jump Game II

Solved

Medium | Topics | Companies

You are given a **0-indexed** array of integers `nums` of length `n`. You are initially positioned at index 0.

Each element `nums[i]` represents the maximum length of a forward jump from index `i`. In other words, if you are at index `i`, you can jump to any index `(i + j)` where:

- `0 <= j <= nums[i]` and
- `i + j < n`

Return *the minimum number of jumps to reach index* `n - 1`. The test cases are generated such that you can reach index `n - 1`.

Example 1:

Code

Java | Auto

```java
        int end = 0;
        int maxReach = 0;

        for (int i = 0; i < nums.length - 1; i++) {

            maxReach = Math.max(maxReach, i + nums[i]);

            if (i == end) {   // time to make a jump
                jumps++;
                end = maxReach;
            }
        }

        return jumps;
    }
}
```

Saved

Ln 20, Col 2

Description | Accepted | Editorial | Solutions | Submissions

49. Group Anagrams

Solved

Medium | Topics | Companies

Given an array of strings `strs`, group the anagrams together. You can return the answer in **any order**.

Example 1:

> **Input:** `strs = ["eat","tea","tan","ate","nat","bat"]`
>
> **Output:** `[["bat"],["nat","tan"],["ate","eat","tea"]]`
>
> **Explanation:**
>
> - There is no string in strs that can be rearranged to form `"bat"`.
> - The strings `"nat"` and `"tan"` are anagrams as they can be rearranged to form each other.

Code

Java | Auto

```java
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {

        Map<String, List<String>> map = new HashMap<>();

        for (String str : strs) {

            char[] arr = str.toCharArray();
            Arrays.sort(arr);                // sort characters
            String key = new String(arr);    // make sorted string as key

            if (!map.containsKey(key)) {
                map.put(key, new ArrayList<>());
            }

            map.get(key).add(str);
        }
```

Saved | Ln 1, Col 1

Description | Accepted | Editorial | Solutions | Submissions

66. Plus One

Solved

Easy | Topics | Companies

You are given a **large integer** represented as an integer array `digits`, where each `digits[i]` is the ith digit of the integer. The digits are ordered from most significant to least significant in left-to-right order. The large integer does not contain any leading `0`'s.

Increment the large integer by one and return *the resulting array of digits*.

Example 1:

```
Input: digits = [1,2,3]
Output: [1,2,4]
Explanation: The array represents the integer 123.
Incrementing by one gives 123 + 1 = 124.
Thus, the result should be [1,2,4].
```

Code

Java | Auto

```java
class Solution {
    public int[] plusOne(int[] digits) {

        for (int i = digits.length - 1; i >= 0; i--) {

            if (digits[i] < 9) {
                digits[i]++;     // add 1
                return digits;   // no carry needed
            }

            digits[i] = 0;       // if 9, make it 0 and carry forward
        }

        // if all digits were 9
        int[] result = new int[digits.length + 1];
        result[0] = 1;

```

Saved

Ln 20, Col

Description | Accepted | Editorial | Solutions | Submissions

73. Set Matrix Zeroes

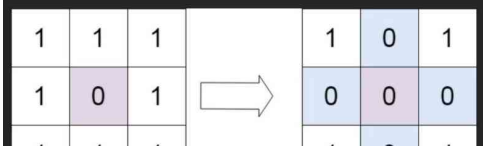
Solved

Medium | Topics | Companies | Hint

Given an `m x n` integer matrix `matrix`, if an element is `0`, set its entire row and column to `0`'s.

You must do it in place.

Example 1:



Code

Java | Auto

```java
class Solution {
    public void setZeroes(int[][] matrix) {

        int m = matrix.length;
        int n = matrix[0].length;

        boolean firstRow = false;
        boolean firstCol = false;

        // Check first row
        for (int j = 0; j < n; j++) {
            if (matrix[0][j] == 0) {
                firstRow = true;
                break;
            }
        }

```

Saved

Ln 59, Col 2

Description | Editorial | Solutions | Submissions

75. Sort Colors

Medium | Topics | Companies | Hint

Given an array `nums` with `n` objects colored red, white, or blue, sort them **in-place** so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers `0`, `1`, and `2` to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1:

```
Input: nums = [2,0,2,1,1,0]
Output: [0,0,1,1,2,2]
```

Example 2:

```
Input: nums = [2,0,1]
Output: [0,1,2]
```

Constraints:

- `n == nums.length`
- `1 <= n <= 300`

Code

Java | Auto

```java
class Solution {
    public void sortColors(int[] nums) {

        int low = 0;
        int mid = 0;
        int high = nums.length - 1;

        while (mid <= high) {

            if (nums[mid] == 0) {
                int temp = nums[low];
                nums[low] = nums[mid];
                nums[mid] = temp;

                low++;
                mid++;
            }
            else if (nums[mid] == 1) {
                mid++;
            }
            else {  // nums[mid] == 2
                int temp = nums[mid];
                nums[mid] = nums[high];
                nums[high] = temp;

                high--;
            }
        }
    }
}
```

Description | Accepted | Editorial | Solutions | Submissions

78. Subsets

Solved

Medium | Topics | Companies

Given an integer array `nums` of **unique** elements, return *all possible subsets (the power set)*.

The solution set **must not** contain duplicate subsets. Return the solution in **any order**.

Example 1:

```
Input: nums = [1,2,3]
Output: [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]
```

Example 2:

```
Input: nums = [0]
Output: [[],[0]]
```

Constraints:

Code

Java | Auto

```java
class Solution {
    public List<List<Integer>> subsets(int[] nums) {

        List<List<Integer>> result = new ArrayList<>();
        backtrack(nums, 0, new ArrayList<>(), result);
        return result;
    }

    private void backtrack(int[] nums, int index,
                           List<Integer> list,
                           List<List<Integer>> result) {

        result.add(new ArrayList<>(list));  // add current subset

        for (int i = index; i < nums.length; i++) {
            list.add(nums[i]);              // choose
            backtrack(nums, i + 1, list, result);
            list.remove(list.size() - 1);   // backtrack
        }
    }
```

Saved

Ln 1, Col 1

Description | Editorial | Solutions | Submissions

79. Word Search

Medium | Topics | Companies

Given an `m x n` grid of characters `board` and a string `word`, return `true` *if* `word` *exists in the grid.*

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

Example 1:

A	B	C	E
S	F	C	S
A	D	E	E

```
Input: board = [["A","B","C","E"],["S","F","C","S"],
["A","D","E","E"]], word = "ABCCED"
Output: true
```

Example 2:

Code

Java | Auto

```java
class Solution {
    public boolean exist(char[][] board, String word) {

        int m = board.length;
        int n = board[0].length;

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (dfs(board, word, i, j, 0)) {
                    return true;
                }
            }
        }

        return false;
    }

    private boolean dfs(char[][] board, String word, int i, int j, int index) {

        if (index == word.length()) return true;

        if (i < 0 || j < 0 ||
            i >= board.length || j >= board[0].length ||
            board[i][j] != word.charAt(index)) {
            return false;
        }

        char temp = board[i][j];
        board[i][j] = '#';  // mark visited

        boolean found = dfs(board, word, i + 1, j, index + 1) ||
```

Saved | Ln 40, Col 2

Array

Submit

Premium

Description | Editorial | Solutions | Submissions

Given an array `nums` of `n` integers, return *an array of all the* ***unique*** *quadruplets* `[nums[a], nums[b], nums[c], nums[d]]` such that:

- `0 <= a, b, c, d < n`
- `a`, `b`, `c`, and `d` are **distinct**.
- `nums[a] + nums[b] + nums[c] + nums[d] == target`

You may return the answer in **any order**.

Example 1:

```
Input: nums = [1,0,-1,0,-2,2], target = 0
Output: [[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]]
```

Example 2:

```
Input: nums = [2,2,2,2,2], target = 8
Output: [[2,2,2,2]]
```

Constraints:

- `1 <= nums.length <= 200`
- `-10^9 <= nums[i] <= 10^9`
- `-10^9 <= target <= 10^9`

Code

Java | Auto

```java
class Solution {
    public List<List<Integer>> fourSum(int[] nums, int target) {
        List<List<Integer>> res = new ArrayList<>();
        Arrays.sort(nums);
        int n = nums.length;

        for (int i = 0; i < n - 3; i++) {
            if (i > 0 && nums[i] == nums[i - 1]) continue;

            for (int j = i + 1; j < n - 2; j++) {
                if (j > i + 1 && nums[j] == nums[j - 1]) continue;

                int l = j + 1, r = n - 1;
                while (l < r) {
                    long sum = (long) nums[i] + nums[j] + nums[l] + nums[r];

                    if (sum == target) {
                        res.add(Arrays.asList(nums[i], nums[j], nums[l], nums[r]));
                        l++; r--;
                        while (l < r && nums[l] == nums[l - 1]) l++;
                        while (l < r && nums[r] == nums[r + 1]) r--;
                    }
                    else if (sum < target) l++;
                    else r--;
                }
            }
        }
        return res;
    }
```

Description | Editorial | Solutions | Submissions

33. Search in Rotated Sorted Array

Medium | Topics | Companies

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is **possibly left rotated** at an unknown index `k` (`1 <= k < nums.length`) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be left rotated by `3` indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the possible rotation and an integer `target`, return *the index of* `target` *if it is in* `nums`*, or* `-1` *if it is not in* `nums`.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

```
Input: nums = [4,5,6,7,0,1,2], target = 0
Output: 4
```

Example 2:

```
Input: nums = [4,5,6,7,0,1,2], target = 3
Output: -1
```

Example 3:

```
Input: nums = [1], target = 0
Output: -1
```

Code

Java | Auto

```java
class Solution {
    public int search(int[] nums, int target) {

        int left = 0, right = nums.length - 1;

        while (left <= right) {

            int mid = left + (right - left) / 2;

            if (nums[mid] == target) return mid;

            // Left half sorted
            if (nums[left] <= nums[mid]) {

                if (target >= nums[left] && target < nums[mid])
                    right = mid - 1;
                else
                    left = mid + 1;
            }
            // Right half sorted
            else {

                if (target > nums[mid] && target <= nums[right])
                    left = mid + 1;
                else
                    right = mid - 1;
            }
        }

        return -1;
    }
```